

IP Course Notes

Stephen Arndt

Instructor: Gérard Cornuéjols

Spring 2025

Contents

1	Basics of Integer Programming	2
1.1	Basic Definitions	2
1.2	Cutting Planes	3
1.3	Branch-and-Bound	3
2	Modeling Problems as IPs	3
2.1	Number Theory	3
2.2	Knapsack Problem	5
2.3	Set Packing Problem	5
2.4	Set Covering Problem	5
2.5	Propositional Logic	6
2.6	Facility Location Problem	6
2.7	Union of Polytopes in $\mathbb{R}^n$	7
2.8	Hamiltonian Cycle Problem	7
3	Polyhedral Theory	8
3.1	Basics	8
3.2	Faces of a Polyhedron	9
3.3	Projection of a Polyhedron	10
3.4	Convex Hulls of Integer Points	10
4	Cuts	11
4.1	Chvátal Cuts	11
4.2	Gomory Cuts	11
4.3	Gap Cuts	12
4.4	Split Cuts	12
4.5	Gomory Mixed-Integer Cuts	13
4.6	Lift-and-Project Algorithm	14
4.7	Lift-and-Project Cuts	15

5	Lifting Valid Inequalities	15
5.1	Liftings	15
5.2	Lifting Functions	16
5.3	Knapsack Problem	16
6	Lagrangian Relaxation	17
6.1	Definitions	17
6.2	Computing the Lagrangian Dual	18
7	Branch-and-Cut Algorithm	19
7.1	Algorithm Framework	19
7.2	Algorithm Implementation	20

1 Basics of Integer Programming

1.1 Basic Definitions

Definition 1.1. A **linear program** is an optimization problem of the form $P = \max\{c^T x : Ax \leq b, x \geq 0\}$. An **integer program** has the same form with the added constraint $x \in \mathbb{Z}^n$. A **binary integer program** has the same form with the added constraint $x \in \{0, 1\}^n$. A **mixed integer linear program** has the same form with $x_S \in \{0, 1\}^{|S|}$ for some subset $S \subseteq [n]$.

Fact 1.2. $LP \in P$ but $IP, BIP, MILP \in NP$ -complete.

If the number of variables n is fixed, all problems are in P .

Definition 1.3 (Totally Unimodular Matrix). Matrix $A \in \mathbb{R}^{m \times n}$ is **totally unimodular** if all square submatrices have determinant $0, \pm 1$.

Theorem 1.4. If A is a totally unimodular matrix, then $P = \{x \in \mathbb{R}^n : Ax \leq b, x \geq 0\}$ is an integral polytope.

Definition 1.5 (Formulation). A **formulation** of an integer program's feasible region $S \subseteq \mathbb{Z}^n$ is a characterization of the form $S = \{x \in \mathbb{Z}^n : Ax \leq b\}$.

For a given integer program, there are **many** different formulations, and some allow much more efficient solving of the IP than others.

Definition 1.6 (Perfect Formulation). A **perfect formulation** of an integer program's feasible region S is a formulation $S = \{x \in \mathbb{Z}^n : Ax \leq b\}$ where $\text{OPT}(P) = \max\{c^T x : Ax \leq b\}$ always has an integral optimum solution.

The above definition is equivalent to saying $P = \{x \in \mathbb{R}^n : Ax \leq b\}$ is an integral polytope assuming S is bounded. If we have a perfect formulation P for the feasible region of an integer program, we can solve the integer program by solving a linear program over P . There are two major issues though, namely that describing P can be hard, and the number of constraints required can be huge.

Definition 1.7 (Extended Formulation). An **extended formulation** of an integer program's feasible region $S = \{x \in \mathbb{Z}^n : Ax \leq b\}$ is an alternative characterization $S = \text{proj}_x(\{(x, y) \in \mathbb{Z}^n : A'x + G'y \leq b'\})$.

Extended formulations can often be much simpler and more compact, leading to more efficient algorithms for solving the IP.

1.2 Cutting Planes

Algorithm: Solve the LP relaxation of the IP. If the solution is integral, done. Otherwise, find a valid inequality for the IP that cuts off the fractional solution. Add to the LP and repeat.

Analysis: Gomory found an implementation of the algorithm which terminates in finite time.

1.3 Branch-and-Bound

Algorithm: We build a tree T , with the root node being the LP relaxation. We maintain a feasible integral solution $\bar{x} \in \mathbb{Z}^n$ – if this is not clear at the start, then we will eventually find one. At a given node, solve the LP relaxation. If the solution x is integral, update $\bar{x} \leftarrow x$ if x has better objective. Otherwise, suppose $k < x_i < k+1$. Branch into two subproblems with the added constraints $x_i \leq k$ and $x_i \geq k+1$. Repeat. We can cut off a branch whenever the optimal LP solution objective value is worse than the current best IP objective value.

Analysis: If the feasible region of the original LP relaxation is bounded, and thus the number of IP feasible solutions is finite, then the algorithm will terminate in finite time.

2 Modeling Problems as IPs

2.1 Number Theory

2.1.1 Computing $\text{gcd}(a, b)$

$\text{gcd}(a, b)$ is the largest $d \in \mathbb{Z}^+$ s.t. $a = dx, b = dy$ for $x, y \in \mathbb{Z}$.

Proposition 2.1. $\text{gcd}(a, b) = \min\{ax + by : ax + by \geq 1, x, y \in \mathbb{Z}\}$.

Proof. Let $d = \text{gcd}(a, b)$ and $m = \min\{ax + by : ax + by \geq 1, x, y \in \mathbb{Z}\}$. Clearly $d \leq m$ because $ax + by$ is a positive multiple of d . To argue $d \geq m$, AFSOC $d < m$. Then $m \nmid a$ or $m \nmid b$. WLOG $m \nmid a$, so $a = mq + r$ where $1 \leq r \leq m-1$. Then $r = a - mq = a - (a\bar{x} + b\bar{y})q$ yielding a feasible solution to the IP, a contradiction by minimality of m . Thus $d \geq m$, so $d = m$. $\square$

Note: $\text{gcd}(a, b)$ can be computed in polynomial time by the Euclidean algorithm.

2.1.2 Hermite Normal Form

Definition 2.2 (Hermite Normal Form). A matrix $A \in \mathbb{R}^{m \times n}$ is in **Hermite Normal Form** if $A = (D \ 0)$ where D is a square nonnegative lower triangular matrix, and $d_{ii} > 0, d_{ij} < d_{ii}$ for $j < i$ (i.e. all entries nonnegative and the diagonal entries are positive and the largest entries in each row).

Definition 2.3 (Unimodular Column Operation). A **unimodular column operation** on a matrix is i) interchanging two columns, ii) multiplying a column by -1 , or iii) adding or subtracting an integer multiple of a column to another.

Theorem 2.4. Every rational matrix $A \in \mathbb{Q}^{m \times n}$ with full row rank can be brought into HNF by a finite sequence of UCOs.

Proof. WLOG $A \in \mathbb{Z}^{m \times n}$ by scaling. We process the rows of A from top-to-bottom. Initially $A = C$, and at the end $A = (D \ 0)$ where A is in HNF. Suppose at an intermediate step $A = \begin{pmatrix} D & 0 \\ B & C \end{pmatrix}$. Using i) and ii), turn the next row of C into $c_{11} \geq c_{12} \geq \dots \geq c_{1k} \geq 0$. Using iii), turn all entries 0 except for one entry (suppose it's c_{11}). Using iii) again, force all entries of B in the row to be in the interval $[0, c_{11})$. Now $(D \ 0)$ can be extended by the first row of $(B \ C)$. $\square$

Note that the full transformation of A into HNF can be represented by the equation $H = AU$. U is a product of unimodular column operations, and thus U is unimodular.

2.1.3 Solving Linear Diophantine Equations

Definition 2.5 (Linear Diophantine Equations). A system of **linear diophantine equations** has the form $S = \{x \in \mathbb{Z}^n : Ax = b\}$ for $A \in \mathbb{Q}^{m \times n}, b \in \mathbb{Q}^m$ and A has full row rank (so $m \leq n$).

Theorem 2.6. The feasible region of LDEs can be fully characterized in polynomial time!

Proof. Via Theorem 2.4, write $H = AU$ where H is in HNF and U is unimodular. Then $Ax = b$ is equivalent to $HU^{-1}x = b$ is equivalent to $Hy = b$ for $y = U^{-1}x$. U is unimodular, so U^{-1} is unimodular. Thus $y \in \mathbb{Z}^n \iff x \in \mathbb{Z}^n$. $Hy = b$ gives $(D \ 0)y = b$, so $y = \begin{pmatrix} D^{-1}b \\ \mathbb{Z}^{n-m} \end{pmatrix}$. If this is non-integral, no solutions. Otherwise, $x = Uy = U_1 D^{-1}b + U_2 \mathbb{Z}^{n-m}$ where U_1, U_2 are the first m , last $n - m$ columns of U . $\square$

2.1.4 Theorems of Alternatives

Theorem 2.7 (Fredholm Alternative). A system of linear equations $Ax = b$ is infeasible iff $\exists u$ s.t. $uA = 0$ and $ub \neq 0$.

Theorem 2.8 (Integer Farkas' Lemma). Let $A \in \mathbb{Q}^{m \times n}$ and $b \in \mathbb{Q}^m$. Then $Ax = b, x \in \mathbb{Z}^n$ is infeasible iff $\exists u$ s.t. $uA \in \mathbb{Z}^m$ and $ub \notin \mathbb{Z}$.

Proof. The backwards implication is easy. For the forwards implication, if $Ax = b, x \in \mathbb{R}^n$ itself is infeasible, then via Theorem 2.7, $\exists u$ s.t. $uA = 0$ and $ub \neq 0$. We can then scale u to get $ub \notin \mathbb{Z}$.

Otherwise, $Ax = b, x \in \mathbb{R}^n$ is feasible but $Ax = b, x \in \mathbb{Z}^n$ is not. WLOG A has full row rank (just throw out redundant equalities). Via Theorem 2.4, we can write $A = HU$ where $H = (D \ 0)$ is in HNF and U is unimodular, and we must have $y = D^{-1}b \notin \mathbb{Z}^m$. Thus $\exists i$ s.t. $y_i \notin \mathbb{Z}$. Pick u as the i 'th row of D^{-1} . Show $uA \in \mathbb{Z}^n$ and $ub \notin \mathbb{Z}$. $\square$

2.2 Knapsack Problem

In most contexts, the “better” formulation is the one with the stronger LP relaxation!

Knapsack Problem: There are two main formulations K, K_C . The objective is $\max \sum_{i=1}^n c_i x_i$. The natural formulation is $K = \{x \in \{0,1\}^n : \sum_{i=1}^n a_i x_i \leq b\}$. The minimal cover formulation is $K_C = \{x \in \{0,1\}^n : \sum_{i \in C} x_i \leq |C| - 1 \ \forall \text{ minimal covers } C\}$, where a minimal cover $C \subseteq [n]$ is a subset of items which exceeds the knapsack capacity, but removal of any item drops below the knapsack capacity.

2.3 Set Packing Problem

Set Packing and MIS Problem: The objective is $\max \sum_{i=1}^m x_i$. The natural formulation is $S = \{x \in \{0,1\}^m : Ax \leq 1\}$ where $A \in \{0,1\}^{n \times m}$. Each row corresponds to an element, and each column corresponds to a set. This can be represented as a **maximum independent set** problem in a graph $G = ([m], E)$, where vertices = sets, and edge $(u, v) \in E$ if there exists some $i \in [n]$ s.t. $i \in S_u$ and $i \in S_v$. Of course, the maximum independent set problem can be represented as a set packing problem.

For a set packing instance, the maximal clique formulation is $S_C = \{x \in \{0,1\}^m : \sum_{i \in K} x_i \leq 1 \ \forall \text{ maximal cliques } K\}$.

Definition 2.9 (Perfect Graph). A **perfect graph** is a graph G where the maximal clique formulation for G is a perfect formulation.

Theorem 2.10 (Strong Perfect Graph Theorem). *G is a perfect graph if and only if G has no odd holes (odd-length induced cycle of length ≥ 5) or odd antiholes (complements of odd holes).*

2.4 Set Covering Problem

Set Covering Problem: The objective is $\min \sum_{i=1}^m x_i$. The natural formulation is $S = \{x \in \{0,1\}^m : Ax \geq 1\}$ where $A \in \{0,1\}^{n \times m}$.

Definition 2.11 (Clutters, Covers, and Blockers). A **clutter** $\mathcal{S} \subseteq \mathcal{P}(E)$ of a set X is a collection of subsets of E s.t. no S_i is contained in any other S_j . A **cover** $T \subseteq E$ of $\mathcal{S}$ is a subset of E s.t. $T \cap S_i \neq \emptyset$ for all $S_i \in \mathcal{S}$. The **blocker** $\mathcal{T} \subseteq \mathcal{P}(E)$ of clutter $\mathcal{S}$ is the collection of minimal covers of $\mathcal{S}$.

Proposition 2.12. *Suppose $\mathcal{S}, \mathcal{T}$ are clutters. If $\mathcal{T}$ is the blocker of $\mathcal{S}$, then $\mathcal{S}$ is the blocker of $\mathcal{T}$.*

Theorem 2.13. *Suppose $\mathcal{S}, \mathcal{T}$ are clutters and blockers. If a formulation $\{x \in \{0, 1\}^E : \sum_{e \in S} x_e \geq 1 \ \forall S \in \mathcal{S}\}$ is perfect, then the formulation $\{x \in \{0, 1\}^E : \sum_{e \in T} x_e \geq 1 \ \forall T \in \mathcal{T}\}$ is perfect.*

Min $s - t$ Cut and Path: Consider $E =$ edges of a graph G with a source s and sink t . Let $\mathcal{S} =$ minimal $s - t$ paths of G . The blocker $\mathcal{T} =$ minimal $s - t$ cuts of G . The min-weight $s - t$ cut problem is $\min\{\sum_{e \in E} w_e x_e : \sum_{e \in P} x_e \geq 1 \ \forall s - t \text{ paths } P, x \in \{0, 1\}^E\}$ and the min-weight $s - t$ path problem is $\min\{\sum_{e \in E} w_e x_e : \sum_{e \in C} x_e \geq 1 \ \forall s - t \text{ cuts } C, x \in \{0, 1\}^E\}$. Both are perfect formulations for integral weights, and the fact that *both* are perfect formulations together can be seen as a consequence of Theorem 2.13.

Min-Cut as Set Cover: $\min\{\sum_{e \in E} w_e x_e : \sum_{e \in T} x_e \geq 1 \ \forall \text{ spanning trees } T, x \in \{0, 1\}^E\}$. This is not a perfect formulation, but there are still polynomial-time algorithms for the min-cut problem.

2.5 Propositional Logic

Definition 2.14 (CNF). A **literal** L_i is a variable s.t. $L_i = T, F$. A **clause** is a disjunction of literals. A logical statement in **CNF** is a conjunction of clauses.

The **satisfiability problem** has input = logical statement in CNF, and output = feasible assignment of literals if one exists. This is equivalent to feasibility of the **generalized set cover problem** $\{x \in \{0, 1\}^n : Ax \geq 1 - n(A)\}$ where A is a $0, \pm 1$ matrix and $n(A)$ is the number of -1 's in each row of A .

The **logical inference problem** has input = premise clauses $\mathcal{P}$ and conclusion clause C , and output = YES / NO on whether $\mathcal{P} \implies C$. Again, we can represent this as an instance of the generalized set cover problem. We can represent the conclusion as $C = \bigvee_{j \in \mathcal{P}} L_j \vee \bigvee_{j \in \mathcal{N}} \neg L_j$. We can represent the feasible region of x 's for the premises as $Ax \geq 1 - n(A), x \in \{0, 1\}^n$ and set objective = $\min \sum_{j \in \mathcal{P}} x_j - \sum_{j \in \mathcal{N}} x_j$. If LP is infeasible, output YES. If LP has objective $\geq 1 - |\mathcal{N}|$, output YES. If LP has objective $= |\mathcal{N}|$, output NO.

2.6 Facility Location Problem

In the **facility location problem**, the input is m clients, n sites where a facility can be opened. c_{ij} is the cost of meeting client i 's demand from facility in location j . f_j is the fixed cost of opening facility j .

The natural formulation has $x_j = 1$ if facility j open, 0 otherwise and $y_{ij} =$ fraction of client i 's demand satisfied from location j .

$$\min \sum_{j=1}^n f_j x_j + \sum_{i,j} c_{ij} y_{ij}$$

$$\begin{aligned}
\text{s.t. } \sum_{j=1}^n y_{ij} &\geq 1 & \forall i \in [m] \\
\sum_{i=1}^m y_{ij} &\leq mx_j & \forall j \in [n] \\
y_{ij} &\geq 0 & \forall i, j \\
x_j &\in \{0, 1\} & \forall j
\end{aligned}$$

It turns out the LP relaxation is horrible because oftentimes $x_j = 1/n$ for all j is optimal, giving no information on which facilities to open. A better formulation is to replace $\sum_{i=1}^m y_{ij} \leq mx_j$ with $y_{ij} \leq x_j$ for all i, j . It turns out that in an actual IP solver, you should use the weak formulation, because it will efficiently identify cuts $y_{ij} \leq x_j$ (and won't need all of them)!

2.7 Union of Polytopes in $\mathbb{R}^n$

Suppose we want to describe the feasible region of the **union** of polytopes $P_i = \{y \in \mathbb{R}^n : A_i y \leq b^i\}$ for $i = 1, 2, \dots, k$.

Big M Representation: There exists some constant $M \geq 0$ s.t. for all $y \in \bigcup P_i$, $A_i y \leq b^i + M\mathbf{1}$. Thus the feasible region of $y \in \bigcup P_i$ can be represented in $\mathbb{Z}^k \times \mathbb{R}^n$ as $\{x \in \{0, 1\}^k, y \in \mathbb{R}^n : A_i y \leq b^i + M\mathbf{1}^T(\mathbf{1} - x_i), \sum_{i=1}^k x_i \geq 1\}$.

Balas Formulation: The feasible region of $y \in \bigcup P_i$ can be represented in $\mathbb{Z}^k \times \mathbb{R}^n$ as $S^* = \{x \in \{0, 1\}^k, y, y_1, \dots, y_k \in \mathbb{R}^n : A_i y^i \leq b^i x_i, \sum_{i=1}^k x_i = 1, \sum_{i=1}^k y_i = y\}$.

Theorem 2.15 (Balas). *$\text{conv}(S^*)$ is S^* with $x \in \{0, 1\}^k$ relaxed to $x \in [0, 1]^k$.*

Proof. Let the relaxed S^* set be S^{**} . Clearly $\text{conv}(S^*) \subseteq S^{**}$. Next, let $(y, y^1, y^2, \dots, y^k, x_1, \dots, x_k) \in S^{**}$ be arbitrary. Decompose S^{**} into $\sum_{i=1}^n x_i z_i$ where z_i has the y -component equal to y^i/x_i , the y^i -component equal to y^i/x_i , the x_i component equal to 1, and all other components equal to 0 (if $x_i = 0$, then don't include a term in the sum). This gives a convex combination of points in S^* , so $S^{**} \subseteq \text{conv}(S^*)$. Thus $\text{conv}(S^*) = S^{**}$. $\square$

2.8 Hamiltonian Cycle Problem

In the **hamiltonian cycle problem**, the input is a directed graph $D = (V, A)$, and the output is a tour using each vertex $i \in V$ exactly once. A natural formulation would have $x_{ij} = 1$ if tour uses arc i, j , 0 otherwise, and would be $\{x \in \{0, 1\}^{|E|} : \sum_j x_{ij} = 1 \forall i, \sum_i x_{ij} = 1 \forall j\}$, imposing the constraints that the collection of edges should have in-degree 1 and out-degree 1 on every node. The issue is that this formulation allows subtours, i.e. collections of directed cycles.

The **Miller-Tucker-Zemlin** (MTZ) Formulation fixes this issue by adding variables u_i representing the position of the node $i \geq 2$ in the tour. 1 is the “home city.” We add constraints $u_i - u_j + 1 \leq n(1 - x_{ij})$ for all $i, j \neq$

1. The **Dantzig-Fulkerson-Johnson** (DFJ) Formulation adds constraints $\sum_{i,j \in S} x_{ij} \leq |S| - 1$ for all S s.t. $2 \leq |S| \leq |V| - 1$. Let P_{MTZ} be the first formulation (projected onto x) and P_{DFJ} be the second formulation. Upon working out P_{MTZ} , it turns out that P_{DFJ} is a stronger formulation.

3 Polyhedral Theory

3.1 Basics

Fourier elimination is an algorithm for solving linear systems by eliminating variables one at a time. The General Theorem of Alternatives states that a linear system S is infeasible iff a trivially infeasible inequality is a consequence of S . Fourier elimination $\implies$ GTA $\implies$ Strong duality of LPs.

Definition 3.1. A cone is **finitely generated** if $C = \text{cone}(x^1, x^2, \dots, x^m)$ for some $x^1, x^2, \dots, x^m \in \mathbb{R}^n$. A cone is **polyhedral** if $C = \{x \in \mathbb{R}^n : Ax \leq 0\}$ for some $A \in \mathbb{R}^{m \times n}$.

Theorem 3.2 (Minkowski). *A cone is polyhedral iff it is finitely generated.*

Definition 3.3. $P \subseteq \mathbb{R}^n$ is a **polyhedron** if $P = \{x \in \mathbb{R}^n : Ax \leq b\}$ for some matrix A and vector b . $P \subseteq \mathbb{R}^n$ is a **polytope** if $P = \text{conv}(x^1, x^2, \dots, x^m)$ for finitely many vectors $x^1, x^2, \dots, x^m \in \mathbb{R}^n$.

Theorem 3.4 (Minkowski-Weyl). *P is a polyhedron iff $P = Q + C$ for some polytope Q and polyhedral cone C .*

Corollary 3.5. *P is a polytope iff P is a bounded polyhedron.*

Let $P = \{x \in \mathbb{R}^n : Ax \leq b\}$ be a nonempty polyhedron. Let $M^= = \{i \in [m] : a^i x = b_i \ \forall x \in P\}$ and $M^< = \{i \in [m] : a^i x < b_i \text{ for some } x \in P\}$. Let $A^=, A^<$ be the rows $M^=, M^<$ of A . $M^=$ are called the **implicit equalities**.

Definition 3.6. The **affine hull** of P is $\text{aff}(P) = \{x \in \mathbb{R}^n : A^=x = b^=\}$.

Definition 3.7. The **dimension** of P is $\dim(P) = \text{maximum number of affinely independent points in } P \text{ minus one}$.

Definition 3.8. $x^0, x^1, \dots, x^k \in \mathbb{R}^n$ are **affinely independent** if $\lambda = 0$ is the only solution to $\sum_{i=0}^k \lambda_i x^i = 0, \sum_{i=0}^k \lambda_i = 0$.

Fact 3.9. $x^0, x^1, \dots, x^k \in \mathbb{R}^n$ are **affinely independent** iff $x^1 - x^0, x^2 - x^0, \dots, x^k - x^0 \in \mathbb{R}^n$ are **linearly independent**.

Proposition 3.10. $\dim(P) = n - \text{rank}(A^=)$.

Definition 3.11. The **recession cone** of $P = \{x \in \mathbb{R}^n : Ax \leq b\}$ is $\text{cone}(P) = \{x \in \mathbb{R}^n : Ax \leq 0\}$. The nonzero vectors in $\text{cone}(P)$ are called the **rays** of $\text{cone}(P)$. A ray is an **extreme ray** if it cannot be written as a convex combination of two distinct rays.

Rays r^1, r^2 are considered distinct if $r^2 \neq \mu r^1$ for all $\mu > 0$.

3.2 Faces of a Polyhedron

Let $P = \{x \in \mathbb{R}^n : Ax \leq b\}$ be a polyhedron with m inequalities. Let $M = [m]$.

Definition 3.12. $F \subseteq P$ is a **face** of P if $F = \{x \in P : \pi x = \pi_0\}$ where $\pi x \leq \pi_0$ is a valid inequality for P .

Proposition 3.13. *If F is a nonempty face of P , then $\exists M^* \subseteq M$ s.t. $F = \{x \in \mathbb{R}^n : a^i x = b^i \ \forall i \in M^*, a^i x \leq b^i \ \forall i \in M \setminus M^*\}$.*

The proof strategy is to note that F is the set of optimal solutions to the LP $\max\{\pi x : Ax \leq b\}$, then get an optimal dual solution y^* and take $M^* = \{i \in M : y_i > 0\}$. Complementary slackness will prove the result.

Definition 3.14 (Redundant Inequality). A **redundant** inequality of the system is an inequality which upon removal from the system, does *not change* the actual polyhedron P . Equivalently, a redundant inequality is a consequence of the *other* inequalities in the system, i.e. a *nonnegative linear combination* of the other inequalities in the system.

Thus there are two equivalent concepts for “redundant inequalities” which are both useful depending on the context.

Proposition 3.15. *The inequalities of $Ax \leq b$ inducing faces of dimension $< \dim P - 1$ are **redundant** in the description of P .*

Proof Strategy. The proof strategy is to get a non-redundant inequality of P and show the induced face F has dimension $\dim P - 1$. Thus we want an inequality $a^r x \leq b^r$ s.t. $\exists x \in \mathbb{R}^n$ where $A^=x = b^=$, $a^i x \leq b^i$ for all $i \in M^{\leq} \setminus \{r\}$, and $a^r x > b^r$ (thus if we removed the inequality, this new point x would become part of P , so the inequality is not redundant). In order to show $F = \{x \in P : a^r x = b^r\}$ has dimension $\dim P - 1$, we want to show F only has 1 more implicit equality than P , i.e. the only implicit equalities of F are $A^=x = b^=$ and $a^r x = b^r$, and then we can apply Proposition 3.10. Thus we need to find a point $x' \in F$ s.t. $A^=x' = b^=$, $a^r x' = b^r$, and $a^i x' < b^i$ for all $i \in M^{\leq} \setminus \{r\}$.

We want to use x to get x' . If we can get a point z where $A^=z = b^=$ and $a^i z < b^i$ for all $i \in M^{\leq}$, then we can move from z to x' and pick x to be the point where $a^r x = b^r$. Indeed, z exists, by finding points $z^i \in P$ which have $a^i z^i < b^i$, and then letting z be a convex combination of the z^i 's. This completes the proof. $\square$

Definition 3.16. A **facet** of P is a nonempty face of dimension $\dim P - 1$.

Proposition 3.17. *For each facet F of P , one of the inequalities of $Ax \leq b$ inducing F is a **non-redundant** inequality.*

The proof idea is to proceed directly from definitions, and then use Fredholm's alternative to argue $\exists y \in \mathbb{R}^n$ s.t. $A^=y = 0$ and $a^r y > 0$, where $r \in M$ is one of the inequalities inducing F . To use Fredholm's alternative, you use the fact that $uA^= = a^r$ has no solution, because a^r does **not** give an implicit

equality. From an interior point $x \in F$, you can move in the direction y and exit P , but still be in the polyhedron $\tilde{P}$ (which is P with the facet-defining inequalities removed).

Fact 3.18. $\alpha x \leq \alpha_0$ and $\beta x \leq \beta_0$ induce the same facet iff $\beta x \leq \beta_0$ is of the form $(\lambda a + uA^=)x \leq \lambda \alpha_0 + ub^=$ for some $\lambda > 0$ and $u \in \mathbb{R}^k$.

Theorem 3.19. If P is full-dimensional, then P has a **unique nonredundant representation** $Ax \leq b$ up to positive scalar multiplication.

Theorem 3.20. If $\dim P = n - k$ for $k \geq 1$, and P has t facets, then $P = \{x \in \mathbb{R}^n : a^i x = b_i \text{ for } i = 1, 2, \dots, k, a^i x \leq b_i \text{ for } i = k + 1, k + 2, \dots, k + t\}$ where $a^i x \leq b_i$ is any inequality inducing facet $F_i, i = 1, 2, \dots, t$.

Stable Set Polytope: Let $\text{STAB}(G) = \text{conv}\{x \in \{0, 1\}^n : x_i + x_j \leq 1 \ \forall (i, j) \in E\}$ be the convex hull of stable / independent sets in graph $G = (V, E)$. Then $\dim \text{STAB}(G) = n$. Further, for all maximal cliques $S \subseteq V$, $\sum_{i \in S} x_i \leq 1$ induces a facet of G . The statements can be proven by finding $n + 1, n$ affinely independent points in the polytope, facet.

3.3 Projection of a Polyhedron

Let $P = \{(x, y) \in \mathbb{R}^{n \times p} : Ax + Gy \leq b\}$.

Definition 3.21. The **projection** of P onto x is $\text{proj}_X(P) = \{x \in \mathbb{R}^n : \exists y \in \mathbb{R}^p \text{ s.t. } Ax + Gy \leq b\}$.

Theorem 3.22 (Projection Theorem). $\text{proj}_X(P) = \{x \in \mathbb{R}^n : (v^t)^T(b - Ax) \geq 0 \ \forall t \in T\}$ where $\{v^t\}_{t \in T}$ are the extreme rays of the **projection cone** $Q = \{v \in \mathbb{R}^m : vG = 0, v \geq 0\}$.

The proof idea is to start with $x \in \text{proj}_X(P)$, and write a sequence of “iff” statements, where Farkas’ Lemma is the key step.

3.4 Convex Hulls of Integer Points

Definition 3.23. Let $S \subseteq \mathbb{R}^n$. Then $\text{conv}(S)$ is the set of all points $x \in \mathbb{R}^n$ which are a convex combination of points in S .

An important aspect as the above definition is that convex combinations must be finite.

Theorem 3.24 (Fundamental Theorem of Integer Programming). Let $P \subseteq \mathbb{R}^n$ be a rational polyhedron and $S = P \cap \mathbb{Z}^n$. Then $\text{conv}(S) \subseteq \mathbb{R}^n$ is a rational polyhedron and $\text{rec}(P) = \text{rec}(\text{conv}(S))$.

If P is not rational, then $\text{conv}(S)$ may not even be a polyhedron. For $P = \{x \in \mathbb{R}^2 : x_1 \leq x_2\sqrt{2}, x_2 \geq 1\}$, $\text{conv}(S)$ has an infinite number of vertices.

4 Cuts

Question: If P is a rational polyhedron and $S = P \cap \mathbb{Z}^n$, then $\text{conv}(S)$ is a rational polyhedron...but how can we find an inequality description of $\text{conv}(S)$?

4.1 Chvátal Cuts

Definition 4.1 (Chvátal Cut). Let $P = \{x \in \mathbb{R}_+^n : Ax \leq b\}$. A **Chvátal cut** of P is $\lfloor uA \rfloor x \leq \lfloor ub \rfloor$ for some $u \in \mathbb{R}_+^m$.

The idea behind a Chvátal cut is that $uAx \leq ub$ is a valid inequality for P for $u \in \mathbb{R}_+^m$, so $\lfloor uA \rfloor x \leq \lfloor ub \rfloor$ is valid because it is a weaker inequality, so $\lfloor uA \rfloor x \leq \lfloor ub \rfloor$ is valid for all integer points in P . The actual Chvátal cut may not be valid for P , but it is valid for all of the integer points.

Definition 4.2 (Chvátal Closure). Let $P = \{x \in \mathbb{R}_+^n : Ax \leq b\}$. The **Chvátal closure** of P is $P' = P \cap \{\text{all Chvátal cuts}\}$.

Theorem 4.3 (Chvátal). Let $P = \{x \in \mathbb{R}_+^n : Ax \leq b\}$ be a rational polyhedron. Let P^k be the k 'th Chvátal closure of P for $k \geq 0$. Then there exists a finite k s.t. $P^k = \text{conv}(S)$.

Definition 4.4 (Chvátal Rank). Let $P = \{x \in \mathbb{R}_+^n : Ax \leq b\}$ be a rational polyhedron. The **Chvátal rank** of P is the minimum $k \geq 1$ s.t. $P^k = \text{conv}(S)$.

Theorem 4.5. Let $P = \{x \in \mathbb{R}_+^n : Ax \leq b\}$ where A, b integral. Then the Chvátal closure of P is $P' = P \cap \{\text{all Chvátal cuts where } v \in [0, 1]^m\}$.

Theorem 4.6. The Chvátal closure of P is a polyhedron.

Proof. Consider all possible inequalities $\lfloor vA \rfloor x \leq \lfloor vb \rfloor$ for $v \in [0, 1]^m$. There are a finite number of possible values for $\lfloor vA \rfloor$ and $\lfloor vb \rfloor$. Thus there are a finite number of distinct Chvátal cuts. $\square$

4.2 Gomory Cuts

Definition 4.7 (Gomory Cut of Equality). A **Gomory cut** of an equality $\sum_{j=1}^n a_j x_j = a_0$ is $\sum_{j=1}^n (a_j - \lfloor a_j \rfloor) x_j \geq a_0 - \lfloor a_0 \rfloor$.

The idea behind the Gomory cut is the following. We can rewrite the equality in the following way.

$$\sum_{j=1}^n (a_j - \lfloor a_j \rfloor) x_j + \sum_{j=1}^n \lfloor a_j \rfloor x_j = a_0$$

$\sum_{j=1}^n (a_j - \lfloor a_j \rfloor) x_j \geq 0$ for all $x \in \mathbb{Z}^n$. However, if $x \in \mathbb{Z}^n$, we can say something stronger: because the second sum $\sum_{j=1}^n \lfloor a_j \rfloor x_j$ is an integer, the first sum $\sum_{j=1}^n (a_j - \lfloor a_j \rfloor) x_j$ must be at least the fractional part of a_0 , i.e. $\sum_{j=1}^n (a_j - \lfloor a_j \rfloor) x_j \geq a_0 - \lfloor a_0 \rfloor$.

Definition 4.8 (Gomory Cut of Polyhedron). Let $P = \{x \in \mathbb{R}_+^n : Ax \leq b\}$ where A, b integral. A **Gomory cut** of P is generated in the following way. First, add slacks to get $Ax + s = b, s \geq 0$. Next, get a linear combination of rows $\lambda Ax + \lambda s = \lambda b$ where $\lambda \in \mathbb{R}^m$. Next, generate a Gomory cut of the equality. Lastly, substitute $s = b - Ax$ to get an inequality in terms of x .

Definition 4.9 (Gomory Closure). Let $P = \{x \in \mathbb{R}_+^n : Ax \leq b\}$. The **Gomory closure** of P is $P' = P \cap \{\text{all Gomory cuts}\}$.

Theorem 4.10. *Let $P \subseteq \mathbb{R}^n$ be a rational polyhedron. Then the Chvátal closure of P is identical to the Gomory closure of P !*

Proof. Given a Chvátal cut $\lfloor uA \rfloor x \leq \lfloor ub \rfloor$ where $0 \leq u < 1$, it is a consequence of $uAx + us = ub$ and the Gomory cut $(uA - \lfloor uA \rfloor)x + us \geq ub - \lfloor ub \rfloor$. Given a Gomory cut $(uA - \lfloor uA \rfloor)x + (u - \lfloor u \rfloor)s \geq ub - \lfloor ub \rfloor$ for $u \in \mathbb{R}^m$, upon plugging in $s = b - Ax$, it is a consequence of P and a corresponding Chvátal cut. $\square$

4.3 Gap Cuts

Definition 4.11 (Gap Cut). Let $P = \{x \in \mathbb{R}_+^n : Ax \leq b\}$ be a rational polyhedron. A **gap cut** is an inequality $\pi x \leq \pi_0$ where π, π_0 integral, and $P \cap \{x \in \mathbb{R}^n : \pi x \geq \pi_0 + 1\} = \emptyset$.

Note that I made up the term “gap cut”.

Proposition 4.12. *Gap cuts and Chvátal cuts are exactly the same.*

Proof. Let $\pi x \leq \pi_0$ be a gap cut. Let $\beta = \max\{\pi x : x \in P\}$. Then $\pi x \leq \beta$ is a valid inequality for P . Thus $\lfloor \pi \rfloor x \leq \lfloor \beta \rfloor$ is a Chvátal cut, which is equivalent to $\pi x \leq \pi_0$.

Let $\lfloor uA \rfloor x \leq \lfloor ub \rfloor$ be a Chvátal cut from the valid inequality $uAx \leq ub$. Then $uAx < \lfloor ub \rfloor + 1$ for all $x \in P$. Thus $\pi = \lfloor uA \rfloor, \pi_0 = \lfloor ub \rfloor$ gives a gap cut for P . $\square$

4.4 Split Cuts

We now focus on mixed integer-linear programs. Let $P = \{(x, y) \in \mathbb{R}_+^{n+p} : Ax + Gy \leq b\}$ for A, G, b rational.

Theorem 4.13 (Fund. Thm. for MILPs). *Let $S = P \cap (\mathbb{Z}^n \times \mathbb{R}^p)$. Then $\text{conv}(S)$ is a rational polyhedron.*

Definition 4.14 (Split Cut). A **split cut** of P is a valid inequality for $\Pi_1 \cup \Pi_2$, where $\Pi_1 = P \cap \{(x, y) : \pi x \leq \pi_0\}$ and $\Pi_2 = P \cap \{(x, y) : \pi x \geq \pi_0 + 1\}$ for $\pi \in \mathbb{Z}^n, \pi_0 \in \mathbb{Z}$.

It is easy to see that $S \subseteq \Pi_1 \cup \Pi_2$.

Definition 4.15 (Split Closure). The **split closure** of P is $P' = P \cap \{\text{all split cuts}\}$.

P' is a polyhedron. However, split closures do not necessarily converge to $\text{conv}(S)$ in finite time. The example is $P = \{(x_1, x_2, y) \in \mathbb{R}_+^3 : x_1 \geq y, x_2 \geq y, x_1 + x_2 + 2y \leq 2\}$. P has a fractional vertex $(1/2, 1/2, 1/2)$, and repeated split closures always leaves a fractional vertex $(1/2, 1/2, t)$ for $t > 0$.

4.5 Gomory Mixed-Integer Cuts

Let $S = \{(x, y) \in \mathbb{Z}_+^n \times \mathbb{R}_+^p : \sum a_j x_j + \sum g_j y_j = b\}$. Let f_0 be the fractional part of b , f_j be the fractional part of a_j for $j \geq 1$. Then we have

$$\sum_{f_j \leq f_0} f_j x_j + \sum_{f_j > f_0} (f_j - 1)x_j + \sum g_j y_j = k + f_0$$

where $k \in \mathbb{Z}$. We know $k \geq 0$ or $k \leq -1$, allowing us to split into two inequalities where one or the other holds. We then combine the inequalities by using the fact that if $a^1 w \geq 1$ or $a^2 w \geq 1$ for $w \geq 0$, then $\max(a^1, a^2)w \geq 1$ (where we take a component-wise maximum). This produces the following cut.

Definition 4.16 (Gomory Mixed-Integer Cut). The **Gomory Mixed-Integer Cut** is

$$\sum_{f_j \leq f_0} \frac{f_j}{f_0} x_j + \sum_{f_j > f_0} \frac{1 - f_j}{1 - f_0} x_j + \sum_{g_j > 0} \frac{g_j}{f_0} y_j + \sum_{g_j < 0} \frac{-g_j}{1 - f_0} y_j \geq 1$$

Upon restriction to the pure integer case, we get the following.

Definition 4.17 (GMI Cut for Pure Integer). The **GMI Cut for Pure Integer** case is

$$\sum_{f_j \leq f_0} f_j x_j + \sum_{f_j > f_0} \frac{f_0}{1 - f_0} (1 - f_j) x_j \geq f_0$$

Fact 4.18. *The GMI Cut for Pure Integer case dominates the Gomory cut.*

This is because the Gomory cut is $\sum_{f_j \leq f_0} f_j x_j + \sum_{f_j > f_0} f_j x_j \geq f_0$, and when $f_j > f_0$, we have $\frac{f_0}{1 - f_0} (1 - f_j) < f_0 < f_j$.

As before, given $P = \{(x, y) \in \mathbb{R}_+^{n+p} : Ax + Gy \leq b\}$ where A, G, b rational, we can add slacks to get $Ax + Gy + s = b$ for $s \geq 0$, get a linear combination of the rows $\lambda Ax + \lambda Gy + \lambda s = \lambda b$ for λ unrestricted, generate a GMI cut, and substitute $s = b - Ax - Gy$ back to get an inequality in terms of only x, y .

Definition 4.19 (GMI Closure). The **GMI Closure** of P is $P' = P \cap \{\text{all GMI cuts}\}$.

Theorem 4.20. *The split closure of P is identical to the GMI closure of P !*

4.6 Lift-and-Project Algorithm

The Lift-and-Project algorithm is an algorithm for computing $\text{conv}(S)$ where $S = \{x \in \{0, 1\}^n \times \mathbb{R}^p : Ax \geq b\}$ is a mixed 0, 1 program from its LP relaxation $P_0 = \{x \in \mathbb{R}^{n+p} : Ax \geq b\}$. The algorithm involves repeated application of the Lift-and-Project operation, which maps polyhedra to polyhedra. The Lift-and-Project operation starts with polyhedron $P = \{x \in \mathbb{R}^{n+p} : Ax \geq b\}$ and integer $j \in [n]$. Then,

1. Generate the nonlinear system $x_j(Ax - b) \geq 0, (1 - x_j)(Ax - b) \geq 0$.
2. Linearize the system by substituting y_i for $x_i x_j, i \neq j$ and x_j for x_j^2 . Call the resulting polyhedron M_j .
3. Project M_j onto the x -space and call the resulting polyhedron P_j . Output P_j .

The Lift-and-Project algorithm starts with P^0 , and sequentially performs the Lift-and-Project operation for $j = 1, 2, \dots, n$.

Theorem 4.21 (Balas). *The Lift-and-Project algorithm computes $\text{conv}(S)$.*

Let $S = \{x \in \{0, 1\}^n \times \mathbb{R}^p : Ax \geq b\}$ and $P = \{x \in \mathbb{R}^{n+p} : Ax \geq b\}$. Let $j \in [n]$ be arbitrary and P_j be the output of the Lift-and-Project operation on P .

Lemma 4.22. $S \subseteq P_j \subseteq P$.

Proof Idea. It is easy to show $S \subseteq P_j$, and then to show $P_j \subseteq P$, you can show that in $M_j, x_j(Ax - b) + (1 - x_j)(Ax - b) \geq 0$ after linearization gives $Ax \geq b$. $\square$

Theorem 4.23. $P_j = \text{conv}\{(P \cap \{x_j = 0\}) \cup (P \cap \{x_j = 1\})\}$.

The proof is pretty technical, so I'll skip it.

Proof of Theorem 4.21. We proceed by induction on n . Let $S_t = \{x \in \{0, 1\}^t \times \mathbb{R}^{n-t+p} : Ax \geq b\}$ for $t \geq 0$. Let P_t be the output of the Lift-and-Project operation on P_{t-1} for $t \geq 1$. We will show that $P_t = \text{conv}(S_t)$.

For $t = 0$, the claim holds. Now assume it holds for $t - 1$ for $t \geq 1$, i.e. $P_{t-1} = \text{conv}(S_{t-1})$. We want to show $P_t = \text{conv}(S_t)$. Via Theorem 4.23, we have

$$\begin{aligned}
 P_t &= \text{conv}\{(P_{t-1} \cap \{x_t = 0\}) \cup (P_{t-1} \cap \{x_t = 1\})\} \\
 &= \text{conv}\{(\text{conv}(S_{t-1}) \cap \{x_t = 0\}) \cup (\text{conv}(S_{t-1}) \cap \{x_t = 1\})\} \\
 &=^? \text{conv}\{(\text{conv}(S_{t-1} \cap \{x_t = 0\}) \cup (\text{conv}(S_{t-1} \cap \{x_t = 1\}))\} \\
 &= \text{conv}((S_{t-1} \cap \{x_t = 0\}) \cup (S_{t-1} \cap \{x_t = 1\})) \\
 &= \text{conv}(S_t)
 \end{aligned}$$

The $=^?$ is the main unjustified step. Note that in the fourth equality, we use the fact that $\text{conv}(\text{conv}(A) \cup \text{conv}(B)) = \text{conv}(A \cup B)$. To resolve the $=^?$ step, it turns out that such an equality is not true in general, but it is true because $x_t = 0$ is a face of $\text{conv}(S_{t-1})$ and $x_t = 1$ is a face of $\text{conv}(S_{t-1})$. $\square$

4.7 Lift-and-Project Cuts

In practice, we may not want to actually do the lift-and-project operation, because it is expensive. Instead, we can use the ideas of the lift-and-project operation to produce a **cut generation LP**, which will produce a **single cut** (i.e. a hyperplane) separating some point $\bar{x} \in P \setminus P_j$ from P_j . We can then sequentially generate cuts (i.e. hyperplanes), and add them to our LP relaxation, until we get to $\text{conv}(S)$.

High-Level Description: We now give a high-level description of how to derive the cut generation LP. P_j is a projection of M_j , so we can use Theorem 3.22 to get an inequality description of P_j . The cuts are valid inequalities for P_j , which we derive by nonnegative linear combinations of the inequalities in the description of P_j . If we have a valid inequality $\alpha x \leq \beta$ for P_j , we want $\alpha \bar{x} > \beta$, and conceptually we want to **maximize** the violation $\alpha \bar{x} - \beta$. The final important consideration is that if $\alpha x \leq \beta$ is valid for P_j , then so is $1000\alpha x \leq 1000\beta$, and so pure maximization of the violation leads to unbounded objective – we need to add a **normalization constraint** on α, β (something like the sum of the components equals 1). This allows us to produce our cut generation LP. A Lift-and-Project cut is obtained by solving this LP and finding optimal α, β .

5 Lifting Valid Inequalities

5.1 Liftings

Proposition 5.1. *Let $S \subseteq \{0, 1\}^n$ be arbitrary. Let $S_0 = S \cap \{x \in \mathbb{Z}^n : x_n = 0\}$, $S_1 = S \cap \{x \in \mathbb{Z}^n : x_n = 1\}$. Assume $S_1 \neq \emptyset$ and $\dim(\text{conv}(S_0)) = n - 1$. Let $\sum_{j=1}^{n-1} \pi_j x_j \leq \pi_0$ define a facet of $\text{conv}(S_0)$. Let $\pi_n = \pi_0 - \max\{\sum_{j=1}^{n-1} \pi_j x_j : x \in S_1\}$. Then $\sum_{j=1}^n \pi_j x_j \leq \pi_0$ defines a facet of $\text{conv}(S)$.*

Proof. $\pi x \leq \pi_0$ is valid for S_0, S_1 and is thus valid for S . In order to show $\pi x \leq \pi_0$ defines a facet of $\text{conv}(S)$, we want to show that 1) $\dim(\text{conv}(S)) = n$, and 2) $\dim(F) = n - 1$ where $F = \{x \in \text{conv}(S) : \pi x = \pi_0\}$. We know $\dim(\text{conv}(S_0)) = n - 1$ so $\text{conv}(S_0)$ has n affinely independent points, and because $S_1 \neq \emptyset$ any point in S_1 will grow this set to $n + 1$ affinely independent points, so $\dim(S) = n$. Further $\dim(F \cap \text{conv}(S_0)) = n - 2$, so $F \cap \text{conv}(S_0)$ has $n - 1$ affinely independent points. The point in S_1 maximizing $\sum_{j=1}^{n-1} \pi_j x_j$ will be in F , and it will be affinely independent with the other points, giving n affinely independent points in F . Thus $\dim(F) = n - 1$. $\square$

We can use Proposition 5.1 sequentially. We could have a facet-defining inequality for some subset of variables $J \subseteq [n]$, and sequentially lift the facet-defining inequality to all variables $[n]$. Note that different orders of lifting typically results in different lifting coefficients.

A **lifting** of a facet-defining inequality for $J \subseteq [n]$ is defined as a facet-defining inequality for $[n]$ where the coefficients of $J \subseteq [n]$ are the same. Note that not all liftings can be found sequentially.

5.2 Lifting Functions

Let $S = \{x \in \{0, 1\}^n : Ax \leq b\}$. Assume $A \geq 0, \dim S = n$. Let $\sum_{j \in C} \pi_j x_j \leq \pi_0$ be a valid inequality for $S \cap \{x \in \mathbb{R}^n : x_j = 0 \ \forall j \notin C\}$. We want to lift into a valid inequality $\sum_{j=1}^n \pi_j x_j \leq \pi_0$ for S .

Definition 5.2 (Lifting Function). Let $f : [0, b] \rightarrow \mathbb{R}$ be $f(z) = \pi_0 - \max\{\sum_{j \in C} \pi_j x_j : \sum_{j \in C} a^j x_j \leq b - z, x^C \in \{0, 1\}^{|C|}\}$.

We know that $\pi_j = f(a^j)$ for all $j \notin C$ where π_j is computed according to Proposition 5.1. To see why, it suffices to show $\max\{\sum_{h \in C} \pi_h x_h : (x^C, x_j = 1) \in S^{C \cup \{j\}}\} = \max\{\sum_{h \in C} \pi_h x_h : \sum_{h \in C} a^h x_h + a^j \leq b, x^C \in S^C\}$. Given a feasible solution to one side, you can produce a feasible solution to the other side with the same objective. This also means that if you pick $\pi'_j \leq f(a^j)$, you will get a valid inequality.

Definition 5.3 (Superadditive Function). A function $g : D \rightarrow \mathbb{R}$ is **superadditive** if $g(u + v) \geq g(u) + g(v)$ for all $u, v \in D$ s.t. $u + v \in D$.

Theorem 5.4. Let f be the lifting function and g be a superadditive function s.t. $g \leq f$. Then $\sum_{i=1}^n \pi_i x_i \leq \pi_0$ is valid for S if we pick $\pi_j = g(a^j)$ for all $j \notin C$. Further, if f is superadditive, picking $\pi_j = f(a^j)$ for all $j \notin C$ produces the unique maximal lifting of the valid inequality.

Proof Idea. Pick $\pi_j = g(a^j)$ for all $j \notin C$ and proceed by induction on the number of variables in the valid inequality. We want to show $\pi_j = g(a^j) \leq f_i(a^j)$ for all $j \notin C$, where the f_i 's are the sequential lifting functions generated by the induction. It seems that you proceed more or less directly from here. $\square$

5.3 Knapsack Problem

Let $S = \{x \in \{0, 1\}^n : \sum_{j=1}^n a_j x_j \leq b\}$. WLOG $a_j \geq 0$ for all j (for $a_j < 0$ replace $x_j \leftarrow 1 - x_j$) and $b \geq a_n \geq \dots \geq a_1$. For $J \subseteq [n]$, let x^J be the characteristic vector of J .

Proposition 5.5. Let $C \subseteq [n]$ be a cover. The cover inequality $\sum_{j \in C} x_j \leq |C| - 1$ is **facet-defining** for $\text{conv}(S_0)$ where $S_0 = \text{conv}(S) \cap \{x \in \mathbb{R}^n : x_j = 0 \ \forall j \notin C\}$ if and only if C is a **minimal cover**.

Proof. First, note $\dim(S_0) = |C|$, because S_0 contains $|C| + 1$ affinely independent points $0, e^i$ for all $i \in C$. Let $F = \{x \in S_0 : \sum_{j \in C} x_j = |C| - 1\}$.

If C is a minimal cover, then $x^{J \setminus i} \in F$ for all $i \in C$. This gives $|C|$ affinely independent points in F , so $\dim F = |C| - 1$ and F is a facet of S_0 .

If C is not a minimal cover, then $\sum_{j \in C} x_j \leq |C| - 1$ can be written as the sum of a minimal cover inequality for $C' \subseteq C$ and $x_j \leq 1$ for $j \in C \setminus C'$. Thus C 's cover inequality can be written as a nonnegative linear combination of some other cover inequalities, so it is redundant in the description of S_0 , and thus not facet-defining. $\square$

We can use Proposition 5.5 to generate facet-defining inequalities for $\text{conv}(S)$ by starting with a minimal cover inequality and sequentially lifting it to all variables. Proposition 5.6 gives a simple recipe for determining the lifting coefficients.

Proposition 5.6. *Let $C \subseteq [n]$ be a minimal cover. WLOG $C = \{1, 2, \dots, r\}$ (by relabeling the elements) and $a_1 \geq a_2 \geq \dots \geq a_r$. Let $\mu_h = \sum_{k=1}^h a_k$ for $h = 0, 1, \dots, r$ and $\lambda = \mu_r - b > 0$. If the lifted inequality $\sum_{j \in C} x_j + \sum_{j \notin C} \pi_j x_j \leq |C| - 1$ (*) is facet-defining for $\text{conv}(S)$, then $\forall j \notin C$,*

1. *If $\mu_h \leq a_j \leq \mu_{h+1} - \lambda$, then $\pi_j = h$.*
2. *If $\mu_{h+1} - \lambda < a_j < \mu_{h+1}$, then $h \leq \pi_j \leq h + 1$.*

Further, if 2) holds for some $j \notin C$, at least one facet of the form () exists s.t. $\pi_j = h + 1$.*

Thus we can use Proposition 5.6 to basically determine the lifting coefficients, with a small amount of ambiguity over whether $\pi_j = h$ or $\pi_j = h + 1$ in the second case, which can be resolved by directly computing the lifting coefficients.

We could also generate cover inequalities by using the lifting function $f(z) = \pi_0 - \max\{\sum_{j \in C} x_j : a_j x_j \leq b - z, x^C \in \{0, 1\}^C\} = h - 1, h$ if $\mu_h - \lambda < z \leq \mu_{h+1} - \lambda$. f is not superadditive in general, but you could define a superadditive function $g \leq f$ and use g to generate cover inequalities. This can allow you to generate facets which cannot be generated by sequential lifting. For sake of time, I did not include a detailed example here.

6 Lagrangian Relaxation

6.1 Definitions

Let $z_{\text{IP}} = \max\{c^T x : A_1 x \leq b^1, A_2 x \leq b^2, x \in \mathbb{Z}_+^p \times \mathbb{R}_+^q\}$. $A_1 x \leq b^1$ are “complicating” constraints, $A_2 x \leq b^2$ are “nice” constraints. Assume all data are rational.

Definition 6.1. An MP $\max\{z_R(x) : x \in Q\}$ is a **relaxation** of MP $\max\{z(x) : x \in S\}$ if $S \subseteq Q$ and $\forall x \in S, z_R(x) \geq z(x)$.

Definition 6.2. Let $Q = \{x \in \mathbb{Z}_+^p \times \mathbb{R}_+^q : A_2 x \leq b^2\}$. The **Lagrangian relaxation** of z_{IP} w.r.t. Q is $z_{\text{LR}}(\lambda) = \max\{c^T x + \lambda^T (b^1 - A_1 x) : x \in Q\}$ for $\lambda \in \mathbb{R}_+^{m_1}$.

It is easy to verify that the Lagrangian relaxation is indeed a relaxation of z_{IP} for all $\lambda \in \mathbb{R}_+^{m_1}$. Thus $z_{\text{LR}}(\lambda) \geq z_{\text{IP}}$ for all $\lambda \geq 0$.

Definition 6.3. The **Lagrangian dual** of z_{IP} w.r.t. Q is $z_{\text{LD}} = \min_{\lambda \geq 0} z_{\text{LR}}(\lambda)$.

Theorem 6.4. $z_{\text{LD}} = \max\{c^T x : A_1 x \leq b^1, x \in \text{conv}(Q), x \geq 0\}$.¹

¹Assuming that the shown optimization problem is feasible.

Proof. Let $Cx \leq d, x \geq 0$ be a linear system describing $\text{conv}(Q)$. $z_{\text{LR}}(\lambda) = \max\{c^T x + \lambda(b^1 - A_1 x) : Cx \leq d, x \geq 0\} = \min\{d^T \mu + \lambda^T b^1 : C^T \mu \geq c - A_1^T \lambda, \mu \geq 0\}$. Thus $z_{\text{LD}} = \min\{d^T \mu + \lambda^T b^1 : C^T \mu \geq c - A_1^T \lambda, \mu \geq 0, \lambda \geq 0\}$ where minimization is over λ, μ . Take the LP dual again to get $z_{\text{LD}} = \max\{c^T x : Cx \leq d, A_1 x \leq b^1, x \geq 0\} = \max\{c^T x : A_1 x \leq b^1, x \in \text{conv}(Q), x \geq 0\}$. $\square$

I believe the above proof has some redundancy with including $x \in \text{conv}(Q)$ and $x \geq 0$ (since $x \geq 0$ is a consequence of $x \in \text{conv}(Q)$) but it should still work. The takeaway is that we could relax z_{IP} to $z_{\text{LP}} = \max\{c^T x : A_1 x \leq b^1, A_2 x \leq b^2\}$, but this might be a weak relaxation. The Lagrangian dual allows us to perfectly convexify a subset of constraints. Note that this takes some extra work because we need to work with an inequality description $Cx \leq d, x \geq 0$ of $\text{conv}(Q)$, which is why we would like these constraints to be “nice”, i.e. have a small inequality description of $\text{conv}(Q)$.

6.2 Computing the Lagrangian Dual

Let's now study $\text{conv}(Q)$. Let $\{v^k\}_{k \in K}$ be the extreme points of $\text{conv}(Q)$ and $\{r^j\}_{j \in J}$ be the extreme rays of $\text{conv}(Q)$. $z_{\text{LR}}(\lambda)$ is finite iff $\lambda A_1 r^j \geq c^T r^j$ for all $j \in J$. If that holds, then $z_{\text{LR}}(\lambda) = \lambda b^1 + \max_{k \in K} \{(c - \lambda A^1) v^k\}$. Thus $z_{\text{LR}}(\lambda)$ is a piecewise linear function of λ , so $z_{\text{LR}}(\lambda)$ is a convex nondifferentiable function of λ .

Let $P = \{\lambda \in \mathbb{R}_+^{m_1} : \lambda A_1 r^j \geq c^T r^j \ \forall j \in J\}$ be the polyhedral cone of valid λ for the Lagrangian relaxation. Then $z_{\text{LD}} = \min_{\lambda \in P} z_{\text{LR}}(\lambda)$. Thus we want to minimize a convex nondifferentiable function over a polyhedron.

We can use the subgradient algorithm from convex optimization to do this. Let's recall some definitions and the subgradient algorithm.

Definition 6.5. Let $g : \mathbb{R}^n \rightarrow \mathbb{R}$ be a convex function. A **subgradient** of g at $x^* \in \mathbb{R}^n$ is a direction $s \in \mathbb{R}^n$ s.t. for all $x \in \mathbb{R}^n$, $g(x) \geq g(x^*) + s^T(x - x^*)$.

Fact 6.6. x^* is a minimizer of g iff $s = 0$ is a subgradient of g at x^* .

The following proposition allows us to easily compute subgradients for z_{LD} .

Proposition 6.7. Let $\lambda^* \in \mathbb{R}_+^{m_1}$ and x^* be the optimum solution to $z_{\text{LR}}(\lambda^*)$. Then $s^* = b^1 - A_1 x^*$ is a subgradient of $z_{\text{LR}}(\lambda)$ at λ^* .

Proof. We want to show that $\forall \lambda \in \mathbb{R}_+^{m_1}, z_{\text{LR}}(\lambda) \geq z_{\text{LR}}(\lambda^*) + s^{*T}(\lambda - \lambda^*)$. Consider x^* as a feasible solution for $z_{\text{LR}}(\lambda)$. It has objective $c^T x^* + \lambda^T(b^1 - A_1 x^*)$. $z_{\text{LR}}(\lambda^*) = c^T x^* + (\lambda^*)^T(b^1 - A_1 x^*)$. It is easy to prove the desired inequality via direct substitution. $\square$

Subgradient Algorithm: Choose a starting point λ^1 and set $t = 1$. Given λ^t with subgradient s^t at λ^t , if $s^t = 0$, then λ^t is optimal and done. Otherwise, let $\lambda^{t+1} = \text{proj}_P(\lambda^t - \alpha_t s^t)$ for some $\alpha_t > 0$. Let $t \leftarrow t + 1$ and repeat.

Let's review some schemes for choosing α_t .

1. **Thm (Poljak):** If $\{\alpha_t\}_{t=1}^{\infty}$ is a divergent series with $\lim_{t \rightarrow \infty} \alpha_t = 0$ (ex. $\alpha_t = 1/t$), the subgradient algorithm converges to the optimum, but very slowly.
2. **Thm (Poljak):** If α_t is a geometric series, the subgradient algorithm converges fast, but not necessarily to the optimum.

There are many other schemes. For example, one could maintain the same step size, until a fixed number of iterations passes where no improvement is made, and then decrease the step size.

Facility Location Problem: See Subsection 2.6 for the Facility Location Problem. It turns out that if we set the “nice” constraints as $y_{ij} \leq x_i, y \geq 0, x \in \{0, 1\}^m$ and define the Lagrangian relaxation, it has a simple closed form solution, and we can get a closed form solution for the Lagrangian dual. The Lagrangian dual equals the LP relaxation because $y_{ij} \leq x_i$ for all i, j forms a totally unimodular matrix.

7 Branch-and-Cut Algorithm

7.1 Algorithm Framework

We first state the input and algorithm data.

- **Problem:** $z_{\text{IP}} = \max\{c^T x : x \in S\}$ (IP or MILP).
- **Algorithm Data:** Subproblems $S_i \subseteq S, \cup_i S_i = S$. Relaxations $R_i \supseteq S_i$ and values $\bar{z}_i = \max\{c^T x : x \in R_i\}$ with solution x^i .

We now give a complete description of the algorithm.

1. **Initialization:** Set of active problems = $\{\text{IP}\}$. $S_0 = S, \bar{z}_0 = +\infty, z_{\text{IP}} = -\infty$.
2. **Termination Condition:** If no active problem remains, stop. The solution x^j that yielded $z_{\text{IP}} = c^T x^j$ is optimal.
3. **Bounding:** Select an active problem S_i and remove it from the list. Solve the LP relaxation $R_i \subseteq S_i$ with value $\bar{z}_i$ and solution x^i . If $\bar{z}_i \leq z_{\text{IP}}$, go to Step 2. If $x^i \in S^i$ and $c^T x^i > z_{\text{IP}}$, let $z_{\text{IP}} = c^T x^i$, delete all problems from the active list with $\bar{z}_i \leq z_{\text{IP}}$, and go to Step 2. If $x^i \notin S_i$, go to Step 4.
4. **Branching / Cutting Decision:** Should cutting planes be generated? If yes, go to Step 5. Otherwise, go to Step 6.
5. **Cut Generation:** Generate cutting planes $\alpha x \geq \beta$ which are violated for x^i but valid for S . Add them to S_i , add S_i back into the list, and go to Step 2.
6. **Branching:** Let $S_{i_1}, S_{i_2}, \dots, S_{i_k}$ be a division of S_i . Add problems $\text{IP}_{i_1}, \text{IP}_{i_2}, \dots, \text{IP}_{i_k}$ to the active list and go to Step 2.

7.2 Algorithm Implementation

Let's talk about each of the components of the algorithm and how they can be implemented efficiently. The primary factor in the efficiency of branch-and-cut algorithm is the **strength of formulation** of the LP relaxations (i.e. strength of bounds). This is because weak bounds don't allow us to prune the enumeration tree, which makes it blow up exponentially in size.

Assuming we have strong formulations, let's talk about efficient implementation of other components of the algorithm.

- **LP Relaxations:** Using dual simplex allows for fast bounds and the ability to prune a problem without reaching optimality.
- **Branching:** Done by adding linear constraints. There are a few standard methods.
 1. **Variable Dichotomy:** Suppose we want to enforce $x_j \in \mathbb{Z}$ but currently $x_j^i \notin \mathbb{Z}$. We branch into two nodes $x_j \leq \lfloor x_j^i \rfloor, x_j \geq \lfloor x_j^i \rfloor + 1$.
 2. **Generalized Upper Bound:** For example, suppose $\sum_{j \in Q} x_j = 1$ and all $x_j \geq 0$. Then for any partition of Q into Q_1, Q_2 , we know $\sum_{j \in Q_1} x_j = 0$ or $\sum_{j \in Q_2} x_j = 0$.
 3. **Generalized Disjunction:** $dx \leq d_0$ or $dx \geq d_0 + 1$ where $(d, d_0) \in \mathbb{Z}^{n+1}$.
- **Node Selection:**
 1. **DFS + Backtracking:** Advantage is that it's easy to re-optimize and find a feasible solution early. However, it also has the disadvantage of getting stuck in a bad part of the tree, so it is not used in practice.
 2. **Best Bound:** Branch on the node with the largest $\bar{z}^i$. The idea is that this node must branch eventually, so it makes sense to branch it now.
 3. **Likely Optimum:** Branch on the node most likely to contain the optimum solution.
- **Branching Variable Selection:**
 1. **User-Specified Priority**
 2. **Penalties:** $x_j^i = \lfloor x_j^i \rfloor + f_j^i$. You can estimate the decrease in objective value on the left child D_j^- and right child D_j^+ by using the reduced costs in simplex algorithm. You can then branch on variable $i = \arg \max_j D_j^- D_j^+$. This method also allows you to estimate the value of the optimum integer solution.
 3. **Strong Branching:** Compute degradations by solving LPs exactly for each variable x_j where x_j^i is fractional, and pick the variable

with the largest degradation. For degradations, you can use some combination of exact + estimate, for example by solving exactly on initialization but estimating later.

There are many more heuristics for the branch-and-cut algorithm!